

05 — Stack Technologiczny

[← Powrót do README](#) (./README.md) | [← ISO 27001](#) (./04-iso27001-compliance.md) | [Następna: Milestones & Roadmap](#) → (./06-milestones-roadmap.md)

Obecny Stack (v1.4.1)

Core Components

Komponent	Technologia	Wersja	Rola	Status
Runtime	Python	3.11/3.12	Application runtime	✓ Aktualna
Web Framework	Flask	3.1.3	HTTP API + Web UI	✓ Aktualna
WSGI Server	Gunicorn	22.0.0	Production server	✓ Aktualna
ORM	SQLAlchemy	2.0.36	Database access	✓ Aktualna
Migrations	Alembic	1.14.1	Schema migrations	✓ Aktualna
Database	PostgreSQL	16	Primary data store	✓ Aktualna
DB Driver	psycopg2-binary	2.9.9	PostgreSQL driver	✓ Aktualna
Cache	Redis	7	Cache + Rate Limiter	✓ Aktualna
Redis Client	redis-py	5.0.8	Redis connection	✓ Aktualna
HTTP Client	Requests	2.33.0	External API calls	✓ Aktualna
MISP Client	PyMISP	2.4.179	MISP integration	✓ Aktualna
Scheduler	schedule	1.2.2	Cron-like jobs	⚠ Prosta
Rate Limiting	Flask-Limiter	3.7.0	API rate limiting	✓ Aktualna
Metrics	prometheus-client	0.20.0	Prometheus metrics	✓ Aktualna
Crypto	cryptography	46.0.6	AES-GCM encryption	✓ Aktualna
Config	python-dotenv	1.0.1	ENV file loading	✓ Aktualna
Reverse Proxy	Nginx	latest	TLS + Routing	✓ Aktualna
Containerization	Docker Compose	3.x	Orchestration	✓ Aktualna
Monitoring		latest	Observability	✓ Aktualna

Komponent	Technologia	Wersja	Rola	Status
	Prometheus + Grafana			

Development & Testing

Narzędzie	Wersja	Rola
pytest	8.3.4	Test runner
pytest-cov	6.0.0	Coverage reporting
pytest-mock	3.14.0	Mock utilities
pytest-env	1.1.5	Environment setup
fakeredis	2.25.1	Redis mock
responses	0.25.3	HTTP mock

Proponowane Zmiany i Uzasadnienie

Zmiany planowane

Zmiana	Milestone	Uzasadnienie	Impact
+ Flask-WTF	M1.4.2	CSRF protection, form validation	Niski
+ Flask-Login	M1.4.2	Session management, auth	Niski
+ argon2-cffi	M1.4.2	Password hashing (argon2id)	Niski
+ PyJWT	M1.4.2	JWT token generation/validation	Niski
+ Flask-RESTX	M1.6.0	OpenAPI/Swagger auto-generation	Średni
+ PyYAML	M1.6.1	Feed config management	Niski
+ APScheduler	M1.6.1	Advanced scheduling (cron, triggers)	Średni
- schedule	M1.6.1	Zastąpione przez APScheduler	—
Migrate → pyproject.toml	M1.6.0	Modern packaging standard	Niski

Decision Matrix: APScheduler vs. schedule vs. Celery

Kryterium	schedule (obecny)	APScheduler	Celery
Cron expressions	✗ Brak	✓ Full cron	✓ Full cron
Dynamic scheduling	✗ Hardcoded	✓ Runtime add/remove	✓ Runtime
Persistence	✗ In-memory	✓ DB/Redis store	✓ Redis/RabbitMQ
Concurrency	✗ Single thread	✓ Thread pool	✓ Multi-worker
Complexity	✓ Minimalna	⚠ Średnia	✗ Wysoka
Overhead	✓ Zero	⚠ Niski	✗ Wysoki (broker)
Team experience	✓ Znane	⚠ Do nauki	✗ Nowe

Decyzja: APScheduler — najlepszy balans między możliwościami a złożonością.

Uzasadnienie odrzucenia Celery:

- Wymaga dedykowanego message broker (RabbitMQ/Redis as broker)
- Overengineering dla 10-15 scheduled jobs
- Celery jest potrzebny przy >100 concurrent tasks — nie nasz przypadek
- Dodatkowa infrastruktura do utrzymania

Decision Matrix: Flask-RESTX vs. Flask-Smorest vs. apispec

Kryterium	Flask-RESTX	Flask-Smorest	apispec
Swagger UI	✓ Built-in	✓ Built-in	✗ Brak
OpenAPI 3.x	⚠ 2.0 default	✓ 3.0 native	✓ 3.0
Decorator-based	✓ Proste	✓ Proste	⚠ Manual
Community	✓ Duża	⚠ Średnia	⚠ Mała
Flask 3.x support	✓ Tak	✓ Tak	✓ Tak
Marshmallow	✗ Własne	✓ Required	✓ Optional

Decyzja: Flask-RESTX — najprostsza integracja, najlepsza dokumentacja, zespół nie zna Marshmallow.

Architektura PostgreSQL 16

Wykorzystywane features

```
-- JSONB dla elastycznego metadata storage
CREATE TABLE indicators (
  metadata JSONB NOT NULL DEFAULT '{}',
  -- JSONB indexing
  CREATE INDEX idx_indicators_metadata ON indicators USING gin (metadata);
);

-- Array types dla tagów
tags TEXT[] NOT NULL DEFAULT '{}',
CREATE INDEX idx_indicators_tags ON indicators USING gin (tags);

-- Full-Text Search (pg_trgm)
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_indicators_value_trgm
  ON indicators USING gin (value gin_trgm_ops);

-- Partial indexes (only active indicators)
CREATE INDEX idx_indicators_active_last_seen
  ON indicators (is_active, last_seen)
  WHERE is_active = true;

-- INET type dla IP addresses
ip_address INET;

-- Triggers dla automatic timestamps
CREATE TRIGGER update_timestamp
  BEFORE UPDATE ON indicators
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

Strategia indeksów

Indeks	Typ	Tabela	Use Case	Status
idx_indicators_value_trgm	GIN (trigram)	indicators	Wyszukiwanie substring	✓
idx_indicators_source	B-tree	indicators	Filtrowanie po źródle	✓
idx_indicators_active	B-tree (partial)	indicators	Aktywne IOC	✓
idx_indicators_metadata	GIN	indicators	JSONB queries	✓
idx_indicators_tags	GIN	indicators	Array contains	✓
unique_indicator	UNIQUE	indicators	Upsert key	✓

Redis 7 Configuration

```
# redis.conf
maxmemory 512mb
maxmemory-policy allkeys-lru

# Persistence (AOF)
appendonly yes
appendfsync everysec
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb

# Security (docelowe)
# requirepass ${REDIS_PASSWORD}
# tls-port 6380
# tls-cert-file /etc/redis/tls/redis.crt
# tls-key-file /etc/redis/tls/redis.key
```

Redis usage breakdown

Use Case	Key Pattern	TTL	Memory ~est
Export cache	export:{fmt}:{hash}	300s	~200MB
Rate limiter	limiter:*	auto	~5MB
Health summary	health:summary	5s	~1KB
Correlation snapshot	correlation:snapshot: {type}	60s	~10MB
Session store (new)	session:{id}	1800s	~50MB

Dependency Management

Obecny: requirements.txt

```
# Problem: flat file, no grouping, no dev/prod separation
Flask==3.1.3
gunicorn==22.0.0
SQLAlchemy==2.0.36
...
pytest==8.3.4 # Dev dependency mixed with prod!
```


Docelowy: pyproject.toml (M1.6.0)

```

[project]
name = "ioc-service"
version = "2.0.0"
requires-python = ">=3.11"
description = "Threat Intelligence Feed Aggregator"

dependencies = [
    "Flask>=3.1,<4",
    "gunicorn>=22,<23",
    "SQLAlchemy>=2.0,<3",
    "alembic>=1.14,<2",
    "psycpg2-binary>=2.9,<3",
    "redis>=5.0,<6",
    "requests>=2.33,<3",
    "pymisp>=2.4,<3",
    "Flask-Limiter>=3.7,<4",
    "prometheus-client>=0.20,<1",
    "cryptography>=46,<47",
    "python-dotenv>=1.0,<2",
    # New (M1.4.2)
    "Flask-WTF>=1.2,<2",
    "Flask-Login>=0.6,<1",
    "argon2-cffi>=23,<24",
    "PyJWT>=2.8,<3",
    # New (M1.6.0)
    "Flask-RESTX>=1.3,<2",
    # New (M1.6.1)
    "APScheduler>=3.10,<4",
    "PyYAML>=6.0,<7",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.3,<9",
    "pytest-cov>=6.0,<7",
    "pytest-mock>=3.14,<4",
    "pytest-env>=1.1,<2",
    "fakeredis>=2.25,<3",
    "responses>=0.25,<1",
    "black>=24,<25",
    "ruff>=0.3,<1",
    "mypy>=1.9,<2",
    "radon>=6.0,<7",
]

[tool.black]
line-length = 100
target-version = ["py311"]

[tool.ruff]
line-length = 100
select = ["E", "F", "I", "UP", "B", "SIM"]

[tool.mypy]
python_version = "3.11"
strict = true
warn_return_any = true

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = "test_*.py"
addopts = "--cov=app --cov-report=term --cov-report=html"

```

Version Compatibility Matrix

Component	Min Version	Max Version	Notes
Python	3.11	3.12	3.13 planned for v2.1
PostgreSQL	15	16	pg_trgm, JSONB required
Redis	6.2	7.x	AOF persistence required
Docker	24.0	latest	Compose V2
Nginx	1.24	latest	TLS 1.3 support
Node.js (frontend)	18 LTS	20 LTS	If frontend tooling needed

Licensing Considerations

Dependency	License	Commercial OK	Notes
Flask	BSD-3	✓	Permissive
SQLAlchemy	MIT	✓	Permissive
PostgreSQL	PostgreSQL License	✓	Permissive
Redis	BSD-3 (SSPL for Redis Ltd)	⚠	Free for self-hosted
PyMISP	BSD-2	✓	Permissive
Grafana	AGPLv3	⚠	Free for internal use
Prometheus	Apache 2.0	✓	Permissive
Nginx	BSD-2	✓	Permissive (open source)

Wszystkie zależności są **kompatybilne** z commercial/internal use.

[← ISO 27001](#) (./04-iso27001-compliance.md) | [Następna: Milestones & Roadmap](#) [→](#) (./06-milestones-roadmap.md)